

SplitTracr: A Flexible Performance Evaluation Tool for Cooperative Inference and Split Computing

Nicholas Bovee*
bovee@rowan.edu
Rowan University
Glassboro, New Jersey, USA

Izhar Ali*
aliizh94@students.rowan.edu
Rowan University
Glassboro, New Jersey, USA

Suraj Bitla
bitlas47@students.rowan.edu
Rowan University
Glassboro, New Jersey, USA

Gopi Krishna Patapanchala
patapa89@rowan.edu
Rowan University
Glassboro, New Jersey, USA

Shen-Shyang Ho
hos@rowan.edu
Rowan University
Glassboro, New Jersey, USA

Abstract

We present SplitTracr, a performance evaluation tool used on deep neural networks (DNNs) to simplify the development overhead of split computation-based DNN models. Usage of such a tool allows the neural network to be dynamically split into component parts at any point. Our tool handles the management of configuration of hardware edge devices, export & injection of forward pass tensors during live tests, dataset handling, metrics & logging in the testing process. We examine the structure, methodology, and design choices we made to create SplitTracr, and review the results of our pre-provided base cases.

CCS Concepts

- **Computing methodologies** → **Machine learning approaches**;
- **Computer systems organization** → *Distributed architectures*.

Keywords

Computational Offloading, Cooperative Inference, Deep Learning, Split Computing, Edge Computing, Machine Learning Tools

ACM Reference Format:

Nicholas Bovee, Izhar Ali, Suraj Bitla, Gopi Krishna Patapanchala, and Shen-Shyang Ho. 2025. SplitTracr: A Flexible Performance Evaluation Tool for Cooperative Inference and Split Computing. In *Companion of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE Companion '25)*, May 5–9, 2025, Toronto, ON, Canada. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3680256.3721971>

1 Introduction

SplitTracr is inspired by previous proposals such as “Edgent” [10], “AIoTBench” [12], and “Edge AIBench” [5], implementing dynamic model partitioning and intermediate result compression for efficient communication between an edge device and a server. We saw the need for a centralized tool that could provide commonality to the variety of split computation research we were performing, and

identified what we feel is the most generally compatible structure to perform it. By targeting compatibility directly with PyTorch [16] and Ultralytics [7] YOLO [17] models, we enable compatibility with a variety of DNN architectures rather than being constrained to a specific model. This simplifies the cross-domain knowledge that developers & researchers must integrate before entering the split computation space, while simultaneously increasing the modularity of a given network under test. Developer time can then be focused on structuring their architecture and approach. As an added benefit, SplitTracr also manages the setup & deployment of its tests and datasets across devices, removing networking concerns & data collation from the list of researcher concerns, such as the implementation of fuzzing or privacy practices to avoid the reconstruction of private data [4] by cloud providers if they are externally operated in production environments.

2 Background

Computation Offloading is the “task of sending computation-intensive application components to a remote server” [1]. Computation offloading techniques at the mobile device level result in resource-consuming tasks, such as video analytics, being transmitted and performed on cloud servers to save energy and balance latency [1]. In typical scenarios, only compression or light pre-processing is done to the input data for maximum efficiency. Traditional offloading contains concerns about privacy, security, and network dropout, particularly when transferring information to external servers [11]. In particular, user data privacy can be leaked during the computation offloading process directly through offloaded data [15].

Cooperative Inference via Split Computing [13] is an application of computation offloading for processing DNN or other machine learning models on comparatively low-compute edge devices. It divides a DNN model into two parts, processing one at the edge device and offloading the remainder to a high-power fog/cloud server. The edge device computes only the initial layers of the model, thus minimizing computational resources and energy consumption, while more intensive computation is offloaded to the remote servers, as shown in Figure 1. By keeping intensive computations to the cloud, a reduced latency, energy consumption, or cost is achieved, while resource utilization on the edge devices is kept to a minimum [20]. The key step is identifying or introducing an ideal

*Both authors contributed equally.



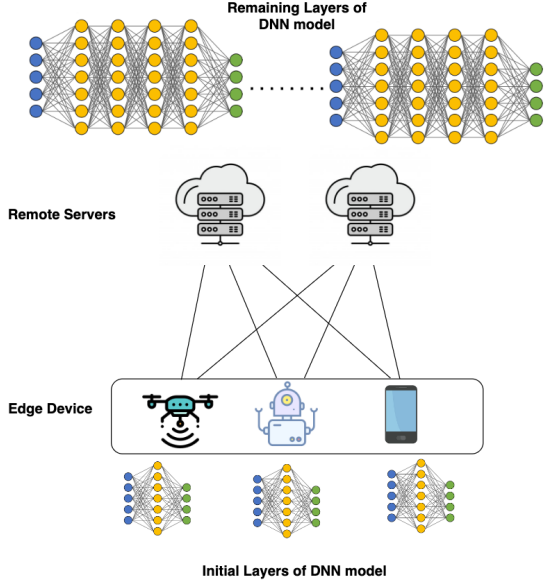


Figure 1: Splitting DNN Models Across Edge Devices and Remote Servers

location to split the model based on key metrics of primary concern. This can be a natural bottleneck in the model or an artificially introduced one [13, 19]. Recent approaches to the split operations also investigate viable techniques in horizontal micro-splits [18], and adaptive pruning [14].

3 SplitTracr Core Functionality

When interacting with a model, SplitTracr implements PyTorch’s [16] hook mechanism to enable dynamic DNN partitioning through forward prehooks and posthooks. During model initialization, hook functions are created and iteratively registered across all submodules based on the test configuration file. Pre-hooks detect split points by monitoring tensor flow at the designated layer index, `model_stop_i`, and managing tensor preprocessing for offloading. Post-hooks capture layer outputs, record runtime metrics, including computation time and memory usage, and prepare tensors for further processing. When a split point is reached, the pre-hook raises a `HookExitException`, halting the forward pass and storing intermediate tensors in a `banked_input` dictionary. These tensors are encapsulated in a `EarlyOutput` wrapper class to ensure they do not extend Python objects that Ultralytics models attempt to handle at re-entry on the cloud side of computation, such as an unwrapped dictionary. A feature of Ultralytics models is that they will begin a training process when passed a dictionary, under the assumption that it is a fully defined training protocol, an undesired outcome for how we control the model flow.

To enable maximum compatibility with existing models, SplitTracr only tracks models through their hook points and no other internal tracking mechanisms. This leads to two compromises that are best illustrated with AlexNet and any given YOLO model. The

first is the non-tracking of common operations that could constitute good split locations, such as NumPy [6] functions like `flatten`. A `flatten` operation is present in AlexNet but is followed by viable split locations immediately after. From this, we conclude that if a NumPy function would be a viable split location, it is an operation that must perform minimal data transformation and therefore would be best executed on the cloud after tensor reinsertion.

The second compromise is in the handling of internally saved tensors, as exemplified by the internal structures of YOLO. During a forward pass, YOLO models save a copy of layer output tensors that will be referenced by concatenation or other layers in the head of the model. These tensors must be passed to the cloud device to ensure correct results, but this increases the amount of data that must be sent. Local collections tracking these are also unavailable for SplitTracr to clone or record, as it has been designed for generic compatibility with any model. To address this, we allow referencing YOLO’s saved layer dictionary while creating our own structure for saved tensor tracking. The saved layer dictionary can be constructed through careful probing and inspection of layer I/O but has not proven necessary in our testing to date. Replicating the internal save structure also ensures forward compatibility with aggressive upload techniques.

3.1 SplitTracr Architecture & Features

SplitTracr employs a distributed client-server architecture built around isolated core subsystems that work together to enable flexible split computing experiments. Each of these is controlled by the master configuration file for a given test, allowing centralized control over the behavior of a given test. SplitTracr’s architecture emphasizes modularity and extensibility through a comprehensive factory-based design pattern. There are three primary extension points that the system is built around: model architecture integration, data pipeline customization, and inference post-processing.

The model registry implements a decorator-based registration system that allows seamless integration of custom architectures while maintaining compatibility with established frameworks. The data management subsystem employs a flexible pipeline architecture centered around PyTorch’s `Dataset` and `DataLoader` abstractions. For inference processing, SplitTracr implements a hierarchical processor system based on abstract interfaces.

3.2 Experiment Manager

Our experiment manager module is the largest of the SplitTracr components, as it covers the largest variety of subcomponents. Command-line interface control, configuration interpretation, implementation and collation of metrics, and control of networking between edge and cloud all fall within its scope. For brevity’s sake, we leave description of the CLI to the repository documentation.

The performance monitoring subsystem provides comprehensive metrics collection across multiple dimensions of system operation. It implements real-time power consumption tracking for both CPU and GPU operations, detailed latency measurements for both computation and communication, memory utilization monitoring, and battery consumption tracking for mobile devices.

The network communication subsystem implements bidirectional TCP socket communication with configurable compression

mechanisms. It utilizes the Blosc2 library [2] for tensor compression, which supports customizable codecs and compression levels. The subsystem includes robust error handling, connection management, and configurable buffer sizes to optimize performance under varying network conditions. This sophisticated communication layer ensures reliable data transfer while minimizing latency and bandwidth usage.

The server component maintains persistent connections and handles concurrent inference requests from multiple clients. Upon receiving compressed tensors, it reconstructs and resumes model execution from the specified layer index (`model_start_i`). Throughout the execution pipeline, the system maintains detailed performance metrics, including host-side execution time, network transfer latency, server-side computation time, and energy consumption from the edge device when available. These metrics provide comprehensive insights into the system’s end-to-end performance for distributed neural network inference.

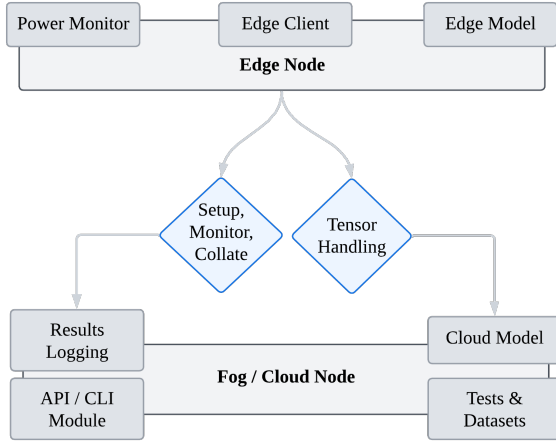


Figure 2: Simplified form of edge & cloud nodes in SplitTracr

3.3 Model Registry

In addition to the previously described implementation of the PyTorch hooks, the model management subsystem implements a comprehensive registry pattern that provides unified access to multiple model sources. This includes direct integration with Torchvision’s model zoo, support for Ultralytics YOLO models, and a decorator-based API for custom architecture registration. The registry handles model initialization, weight management, and architecture-specific optimizations through a unified interface, effectively abstracting the complexities of various deep learning frameworks.

The layer monitoring subsystem leverages PyTorch’s hook mechanism to provide fine-grained control and observation of model execution. Forward pre-hooks handle input preprocessing and metric initialization, whereas post-hooks capture outputs and monitor performance. This dual-hook architecture enables a comprehensive collection of layer-wise metrics, including computation time, memory usage, and power consumption. A dynamic tensor banking

system maintains intermediate results, which are crucial for split computing operations.

3.4 Data Manager

The pipeline subsystem forms the core of data handling and uses an extensible architecture centered on PyTorch’s Dataset and DataLoader abstractions. This subsystem manages data loading, pre-processing, and augmentation through a standardized interface. Advanced features include custom collation functions, multiprocessing worker distribution, and memory-efficient data streaming, enabling researchers to implement domain-specific data processing while maintaining consistent performance monitoring.

4 Evaluation Case Study

Below we examine the performance of SplitTracr-managed tests under their given conditions. Each test file is provided in the tool repository for replication. We evaluated the capabilities of SplitTracr under the following conditions:

- **Edge Device:** Jetson Orin Nano
- **Max Transmission Rate:** 866 Mbps, bidirectional
- **Cloud GPU:** NVIDIA GeForce RTX 4090 with 24GB VRAM
- **Cloud CPU:** AMD Ryzen 7 7700X
- **Python Version:** >3.8, synchronized across devices

4.1 Baseline Replication of Neurosurgeon

Though AlexNet is now far from state-of-the-art, we demonstrate replication of Neurosurgeon’s results for our base case. PyTorch’s implementation diverges from the implementation of AlexNet used by Kang et al. [8] due to optimizations in the structure and training of the network demonstrated by Krizhevsky [9]. Trivial inspection of our results shown in fig. 3 [3] shows that layers 13 & 14, at the end of the features’ nn.Sequential object, are replicated as the optimal location for split. These layers correspond to the MaxPooling and AvgPooling layers in the PyTorch model, each of which condenses data in the forward pass.

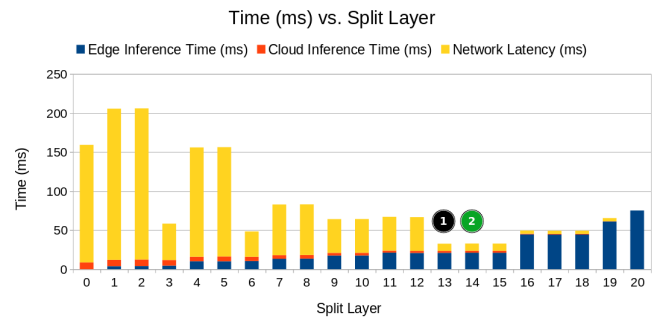


Figure 3: Replication of Kang et al. [8]’s results on AlexNet with the ImageNet dataset. Optimal split layers are marked with 1 & 2, each performing part of AlexNet’s original pooling

4.2 ResNET with ImageNet Classification

We also demonstrate a more modern architecture in our implemented base cases. ResNET-18 is a lightweight implementation of

ResNET, with minimal iterative residual blocks. These results are drawn from singleton images from each ImageNet class, resulting in 1000 images under test. SplitTracr’s default depth for injection of its hooks is above the residual concatenation layer, avoiding separation of skip connections and the accompanying increase in data that would need to be passed to the cloud server. Note that latency per layer in fig. 4 is approximately similar after the initial input layers, and energy consumption displayed in fig. 5 consistently decreases at each layer. These characteristics indicate that the network is best suited to run entirely on the edge device under the given test conditions.

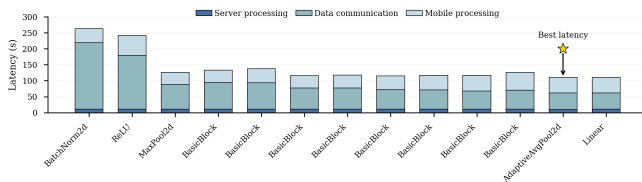


Figure 4: ResNET-18 latency over split indexes. The size of the tensor sent to the cloud is most significant to results.

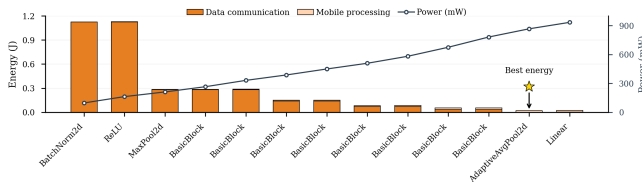


Figure 5: ResNET-18 energy over split indexes. Individual layers are marked in Joules; cumulative cost is in mW.

4.3 YOLOv8s with Custom Dataset

For examining more desirable object detection models in split computation configurations, we provide an implementation of YOLOv8s from Ultralytics. Most YOLO models provided by Ultralytics follow a standard construction pattern, and any of them would be viable for selection in our tool. V8s was chosen for its small size and ability to be run entirely within our edge devices at a reasonable FPS if required. It contains the characteristic feature pyramid, a strong target for optimization in split computation. These results are from our custom Onion Dataset, containing 131 images sourced from split computation precision agriculture work we have performed.

In fig. 6, we see that the optimal split location is extremely early in the network, on our third possible split point. This is corroborated by the results in fig. 7, which target energy consumption as the goal metric rather than latency. The minimum is much more pronounced in the energy cost metric of YOLOv8s, as a convolution is relatively simple for the system to process without significantly increasing output data size. This ideal split point also occurs at the first of several skip connections. These begin data transmission only at the instance of the split. Our results on YOLOv8s are less appealing when considering latency, as expected when profiling a network with long-reaching skip connections in its model architecture.

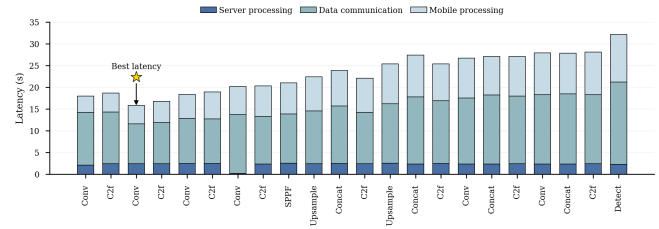


Figure 6: YOLOv8s cumulative layer time cost.

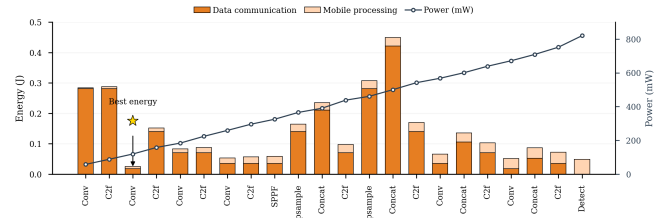


Figure 7: YOLOv8s individual layer mW cost.

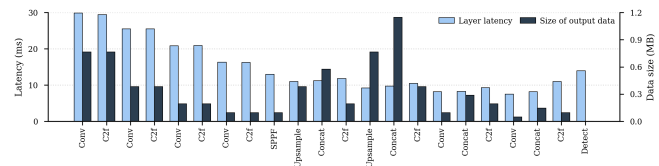


Figure 8: YOLOv8s data analysis

We also include a mapping between individual layer latency and the size of its output data in fig. 8. Generally, these trend toward higher latency with larger output data, with required tensors from previous layers not included in each layer’s bar. Most significantly, these depend on the specific tensor operation in question.

5 Conclusion and Future Work

These examples are quite basic compared to the level of complexity in modern ML models, but we believe they do well to provide insight into the functionality of SplitTracr and pique interest in Cooperative Inference techniques. There are many avenues for improvement in both our tool’s functionality and advancements in the field, despite the current compute-rich environment.

In the future, we aim to improve the real-time performance of the testbed, integrate privacy-preserving techniques, and add support for other common split computation techniques, such as early exit, pruning, and horizontal splitting. We have begun integrating scheduling objects that dynamically control the split layer based on runtime conditions and the live results of the model. This will enhance model pruning for optimal resource efficiency and aid model deployment in production to encourage new users.

For evaluation of SplitTracr, or to contribute to it, the archive repository can be found at <https://doi.org/10.5281/zenodo.15048915>.

Acknowledgments

This research project is funded by NSF Award No. 2128341.

References

- [1] Khadija Akherfi, Micheal Gerndt, and Hamid Harroud. 2018. Mobile cloud computing for computation offloading: Issues and challenges. *Applied computing and informatics* 14, 1 (2018), 1–16.
- [2] Blosc Development Team. 2009–2025. *A fast, compressed and persistent data store library*. <https://blosc.org>.
- [3] Nicholas Bovee, Stephen Piccolo, Shen Shyang Ho, and Ning Wang. 2023. Experimental test-bed for Computation Offloading for Cooperative Inference on Edge Devices. In *EdgeComm: The Fourth Work-shop on Edge Computing and Communications (at ACM/IEEE Symposium on Edge Computing)*. IEEE.
- [4] Matt Fredrikson, Somesh Jha, and Thomas Ristenpart. 2015. Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 1322–1333. doi:10.1145/2810103.2813677
- [5] Tianshu Hao, Yunyou Huang, Xu Wen, Wanling Gao, Fan Zhang, Chen Zheng, Lei Wang, Hainan Ye, Kai Hwang, Zujie Ren, and Jianfeng Zhan. 2019. Edge AIBench: Towards Comprehensive End-to-end Edge Computing Benchmarking. arXiv:1908.01924 [cs.PF]
- [6] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. 2020. Array programming with NumPy. *Nature* 585, 7825 (Sept. 2020), 357–362. doi:10.1038/s41586-020-2649-2
- [7] Glenn Jocher, Jing Qiu, and Ayush Chaurasia. 2023. *Ultralytics YOLO*. <https://github.com/ultralytics/ultralytics>
- [8] Yiping Kang, Johann Hauswald, Cao Gao, Austin Rovinski, Trevor Mudge, Jason Mars, and Lingjia Tang. 2017. Neurosurgeon: Collaborative intelligence between the cloud and mobile edge. *ACM SIGARCH Computer Architecture News* 45, 1 (2017), 615–629.
- [9] Alex Krizhevsky. 2014. One weird trick for parallelizing convolutional neural networks. arXiv:1404.5997 [cs.NE] <https://arxiv.org/abs/1404.5997>
- [10] En Li, Liekang Zeng, Zhi Zhou, and Xu Chen. 2020. Edge AI: On-Demand Accelerating Deep Neural Network Inference via Edge Computing. *IEEE Transactions on Wireless Communications* 19, 1 (2020), 447–457. doi:10.1109/TWC.2019.2946140
- [11] Tianheng Li, Xiaofan He, Siming Jiang, and Juan Liu. 2022. A survey of privacy-preserving offloading methods in mobile-edge computing. *Journal of Network and Computer Applications* 203 (2022), 103395.
- [12] Chunjie Luo, Xiwen He, Jianfeng Zhan, Lei Wang, Wanling Gao, and Jiahui Dai. 2020. Comparison and Benchmarking of AI Models and Frameworks on Mobile Devices. *CoRR* abs/2005.05085 (2020). arXiv:2005.05085
- [13] Yoshitomo Matsubara, Marco Levorato, and Francesco Restuccia. 2022. Split computing and early exiting for deep learning applications: Survey and research challenges. *Comput. Surveys* 55, 5 (2022), 1–30.
- [14] Akrit Mudvari, Antero Vainio, Iason Ofeidis, Sasu Tarkoma, and Leandros Tassiulas. 2024. Adaptive Compression-Aware Split Learning and Inference for Enhanced Network Efficiency. *ACM Trans. Internet Technol.* 24, 4, Article 27 (Nov. 2024), 26 pages. doi:10.1145/3687471
- [15] Xiaoyi Pang, Zhibo Wang, Jingxin Li, Ruiting Zhou, Ju Ren, and Zhetao Li. 2022. Towards online privacy-preserving computation offloading in mobile edge computing. In *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 1179–1188.
- [16] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems* 32, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, and R. Garnett (Eds.). Curran Associates, Inc., 8024–8035.
- [17] J Redmon. 2016. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*.
- [18] Eric Samikwa, Antonio Di Maio, and Torsten Braun. 2024. DISNET: Distributed Micro-Split Deep Learning in Heterogeneous Dynamic IoT. *IEEE Internet of Things Journal* 11, 4 (2024), 6199–6216. doi:10.1109/JIOT.2023.3313514
- [19] Yingchao Wang, Chen Yang, Shulin Lan, Liehuang Zhu, and Yan Zhang. 2024. End-edge-cloud collaborative computing for deep learning: A comprehensive survey. *IEEE Communications Surveys & Tutorials* (2024).
- [20] Ashkan Yousefpour, Neeraj Patil, Gen Ishigaki, et al. 2019. All one needs to know about fog computing and related edge computing paradigms: A complete survey. *Journal of Systems Architecture* 98 (2019), 289–330.